

Recomendadores Generativos I: de los LLMs a la recomendación generativa

IIC3633 – Sistemas Recomendadores

Denis Parra

Profesor Asociado

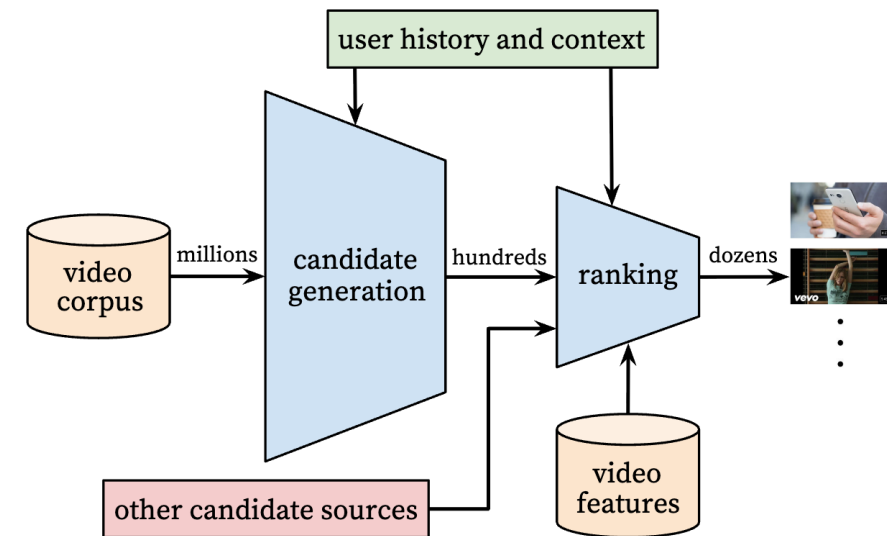
PUC Chile

Contenidos

1. Motivación: de la cascada discriminativa a la generación de ítems
2. Dónde y cómo adaptar LLMs a RecSys (survey de Lin et al.)
3. Beneficios de los LLMs para recomendación
4. LLMs como recomendadores secuenciales: primeros intentos y alineación
5. Objetivos de entrenamiento, inferencia y eficiencia
6. LLMs como recomendadores conversacionales y simuladores de usuario
7. Desafíos y discusión

Pipeline industrial de recomendación

- Sistema en cascada multi-etapa:
Recall → **Pre-ranking** → **Ranking** → **Re-ranking**
- No es solo ingeniería: descompone un problema global de recomendación en problemas locales de puntuación (scoring)
- Consecuencias: pérdida de información entre etapas, objetivos inconsistentes y errores acumulados
- Modelo discriminativo típico: *Embedding* + *MLP* que aprende una función de puntaje sobre (usuario, ítem, contexto)



Tres cuellos de botella de la recomendación discriminativa

Cuello de botella	Problema	Consecuencia
Representación	Embeddings atómicos por ítem (aislamiento semántico)	Cold-start y transferencia entre dominios difíciles
Arquitectura	Componentes fragmentados por etapa; modelos de tamaño modesto	Bajo MFU (<5%) y poco aprovechamiento de scaling
Optimización	Pérdidas específicas por etapa (recall, rank, re-rank)	No se optimiza la distribución global de ítems ni los objetivos de múltiples actores

Cambio de paradigma: recomendar es generar

- **Generative Recommendation (GenRec):** en vez de puntuar candidatos, un modelo generativo produce directamente los identificadores de los ítems
- Nueva estructura del problema: tokenización de ítems → modelo generativo → optimización generativa → salida

Idea guía: *“lo que un LLM no puede generar sobre el comportamiento del usuario, no lo entiende”*

- **Estado actual:** ya hay modelos grandes de lenguaje y de difusión; en recomendación solo hay componentes generativos dentro de recomendadores tradicionales e intentos iniciales de GenRec

Ventajas de GenRec

Dimensión	Ventaja	Ejemplo / dato
Tokenización	Opera a nivel semántico y con vocabulario más compacto	~48,2 M ítems (Amazon Reviews 2023) vs ~200 mil tokens (Llama-4) o ~250 mil (Qwen-3.5)
Arquitectura	Escala en tamaño del modelo; alto MFU; reutiliza tecnologías de NLP	PEFT, razonamiento, agentes
Optimización	Next-token prediction sobre ítems: modela la distribución completa; permite alinear preferencias	Satisfacción del usuario, objetivos de plataforma y ecosistema de creadores

Fuente: Tutorial 'Generative Recommendation: Foundations and Frontiers' (KDD 2026) (pp. 11-13)

MFU: uso efectivo del hardware

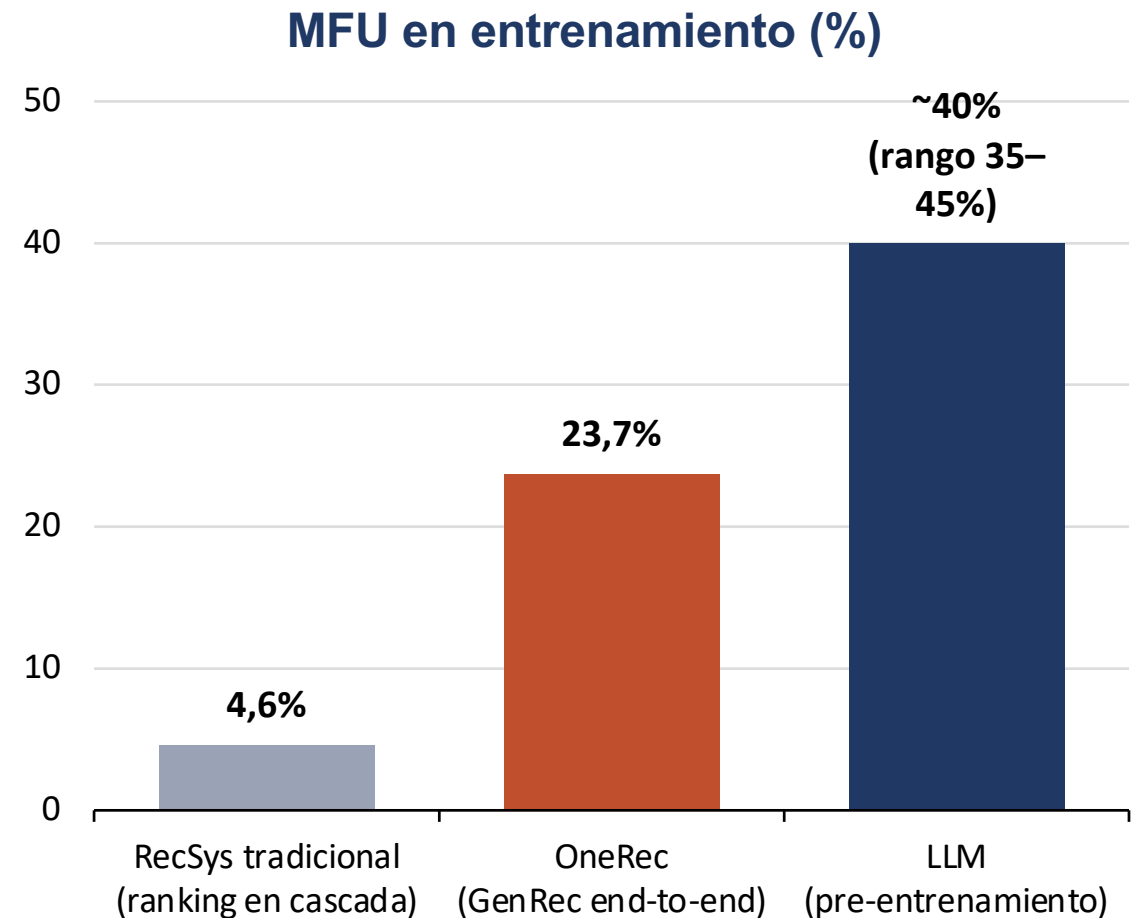
MFU = Model FLOPs Utilization: FLOPs útiles del modelo por segundo ÷ FLOPs max del hardware.

Mide eficiencia, no calidad: un MFU bajo significa que la GPU espera por memoria, comunicación o datos.

RecSys tradicional: tablas de embeddings enormes y etapas fragmentadas dan un MFU de un dígito.

GenRec: un Transformer denso y unificado se acerca al MFU de un LLM, y el cómputo adicional se convierte en calidad (leyes de escalamiento).

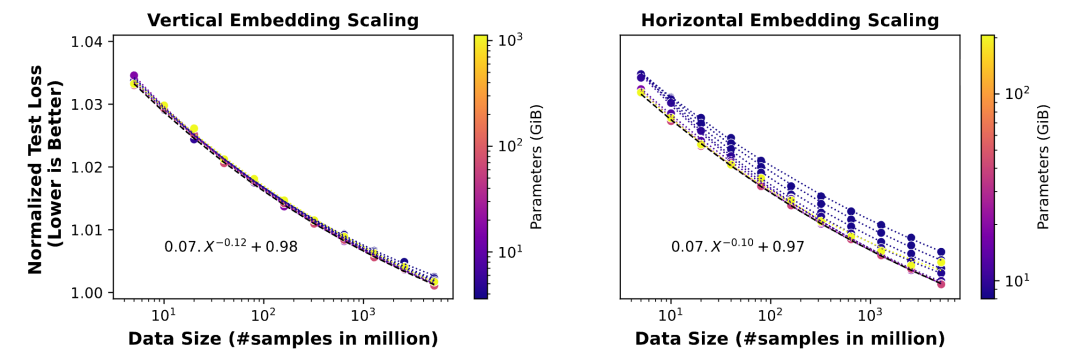
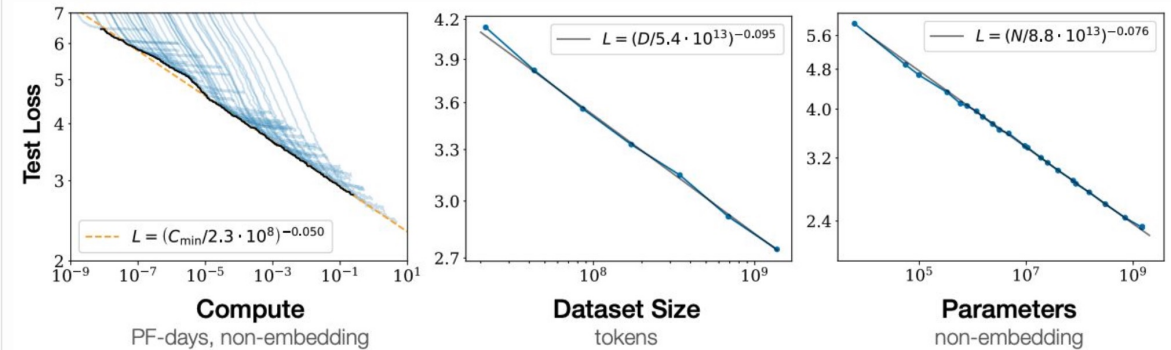
FLOPs: Floating-Point Operations



Fuentes: OneRec Technical Report (Kuaishou, 2025): ranking tradicional 4,6% en entrenamiento y 11,2% en inferencia; OneRec 23,7% y 28,8%. LLMs: Llama 3 405B 38–43% (Meta, 2024), PaLM 46% (Google, 2022); rango típico 35–45%. Tutorial KDD 2026 'Generative Recommendation: Foundations and Frontiers' (pp. 9, 12).

Leyes de escalamiento en recomendación

- Lenguaje: conocimiento del mundo denso, vocabulario de decenas de miles de tokens
- Comportamiento de usuario: interacciones usuario-ítem dispersas, ítems del orden de billones a trillones
- Por eso las leyes de escalamiento rara vez aplican a los recomendadores tradicionales
- **HSTU** (Zhai et al., ICML 2024): primera demostración de scaling law en recomendación; calidad crece como ley de potencias del cómputo; escalado a 1,5 billones de parámetros; +12,4% en métricas online principales



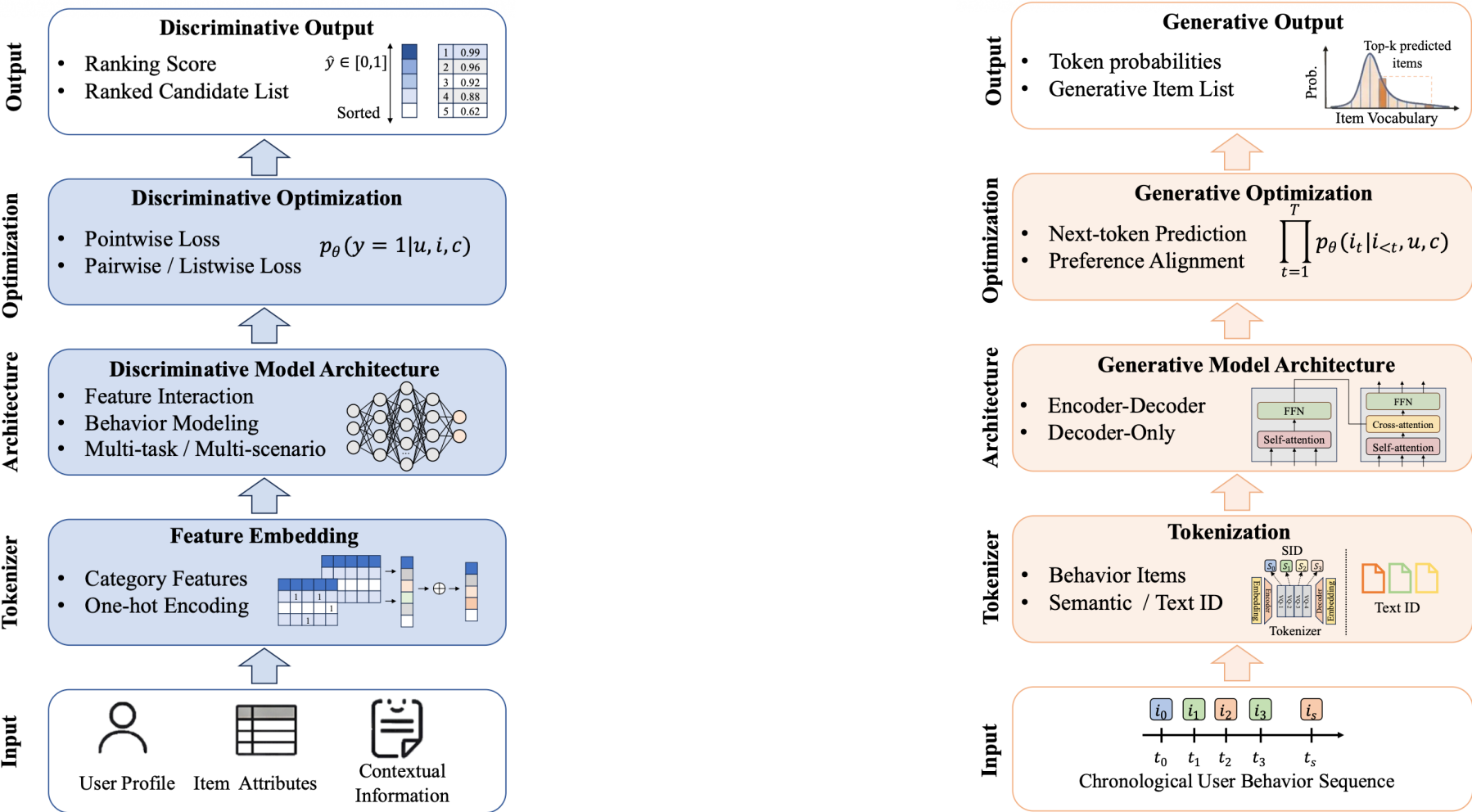
Dos caminos hacia GenRec escalable

- **Adaptar modelos preentrenados:** LLMs adaptados a la tarea de recomendación (ej.: LLaRA, TALLRec) → *esta clase*
- **Entrenar desde cero:**
 - Modelos autoregresivos con Semantic IDs (ej.: TIGER, OneRec) → clases 2, 3 y 4
 - Modelos de difusión → clase 3
- Hilo conductor de las 4 clases (visión *tri-desacoplada*): **Tokenización – Arquitectura – Optimización**

El mismo usuario visto por cuatro recomendadores

UNA INTUICIÓN ANTES DE LOS DETALLES

Dos visiones: Discriminative vs. Generative



El mismo historial, cuatro maneras de recomendar

Modelo	¿Qué es un ítem?	¿Qué calcula el modelo?	¿Cómo sale el top-N?
ALS / MF	un vector aprendido de las interacciones	$u \cdot v$: producto punto usuario-ítem	vecinos más cercanos (ANN)
SASRec	un vector + su posición en la secuencia	self-attention $\rightarrow$ estado h ; luego $h \cdot v$	argmax sobre la tabla de ítems
TALLRec	su título, escrito como texto	el LLM lee el historial y responde Sí/No	una pregunta por candidato
TIGER / LC-Rec	una tupla de tokens: (5, 23, 55)	genera esos tokens uno por uno	beam search sobre el trie

Los dos primeros **puntúan** candidatos que alguien más propuso; los dos últimos **generan** el identificador del ítem. Todo el módulo trata sobre esa diferencia.

Elaboración propia. Los cuatro modelos se detallan en las láminas siguientes.

Recomendar es puntuar: ALS y SASRec

ALS (factorización matricial)

- Entrada: matriz usuario $\times$ ítem. **El orden no se usa**
- Aprende $\mathbf{u} \in \mathbb{R}^{64}$ y $\mathbf{v}_i \in \mathbb{R}^{64}$ por mínimos cuadrados alternados
- Puntaje: $\mathbf{u} \cdot \mathbf{v}(\textit{Arrival}) = 0,82$ vs $\mathbf{u} \cdot \mathbf{v}(\textit{Barbie}) = 0,11$
- El top-10 se arma buscando los vectores más cercanos a $\mathbf{u}$
- Si *Arrival* se estrenó ayer, **no tiene vector**: no puede recomendarla

SASRec (Transformer sobre IDs)

- Entrada: la secuencia **en orden**; cada ítem es una fila de la tabla de embeddings
- Self-attention resume el historial en un estado $\mathbf{h}$
- Puntaje: $\mathbf{h} \cdot \mathbf{v}_i$ para **todos** los ítems del catálogo
- El top-10 es el argmax de esos puntajes
- Mismo problema: $\mathbf{v}_i$ solo se aprende con interacciones

Recomendar es responder: TALLRec

Al usuario le gustaron estas películas:
Matrix, Blade Runner 2049, Inception.

¿Le gustará la película Arrival?
Responde solo Sí o No.

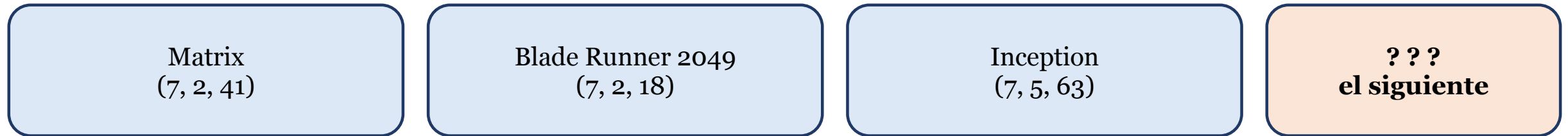
→ **Sí**

El historial completo cabe en un prompt

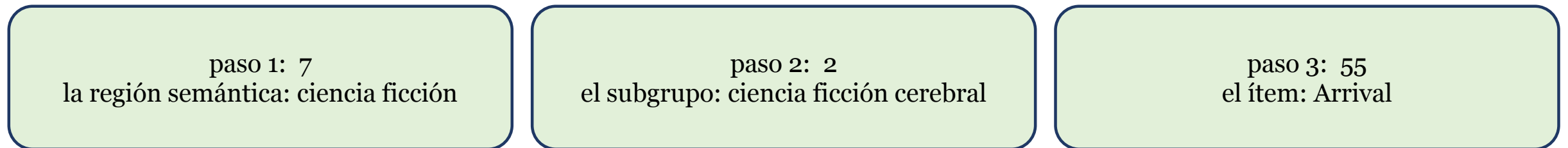
- El ítem se nombra con su **título**: no hay tabla de embeddings que entrenar
- La tarea se vuelve **binaria**; el ranking se arma con la probabilidad del token *Sí*
- Para un top-N hay que **preguntar por cada candidato**: el costo crece con la lista, y por eso suele usarse como *re-ranker* después de un recall tradicional
- Lo que aporta: **conocimiento del mundo**. El LLM ya sabe que esas tres películas son ciencia ficción, y por eso basta con 64 ejemplos de ajuste
- Lo que le falta: **señal colaborativa**, es decir, qué ven juntos los usuarios parecidos

Recomendar es generar: TIGER y LC-Rec

El mismo historial, pero cada película se reemplaza por su Semantic ID



El decoder genera los tokens uno por uno, y una tabla de búsqueda devuelve el ítem:



- No hay producto punto ni índice ANN: el top-10 sale de un **beam search** restringido a identificadores válidos
- *Arrival* tiene Semantic ID desde su sinopsis **aunque nadie la haya visto**: ahí está la promesa de cold-start
- **LC-Rec** aplica esta idea dentro de un LLM: agrega los tokens de ítem al vocabulario y los alinea con el lenguaje

Cuatro recomendadores, cuatro preguntas

Modelo	¿Qué es un ítem?	¿Cómo se obtiene el top-N?	¿Usa el contenido?	Ítem nuevo
ALS / MF	Fila de una tabla de embeddings	Vecinos más cercanos sobre u	No	Sin vector: invisible
SASRec	Fila de una tabla de embeddings	argmax de $h \cdot v_i$	No	Sin vector: invisible
TALLRec	Su título en texto	Una consulta al LLM por candidato	Sí (texto)	Basta con su título
TIGER / LC-Rec	Tupla de tokens semánticos	Beam search sobre los tokens	Sí (texto)	Recibe tokens desde su texto

Un mapa: ¿dónde y cómo adaptar?

LLMS PARA RECOMENDACIÓN

¿Dónde y cómo adaptar LLMs a RecSys?

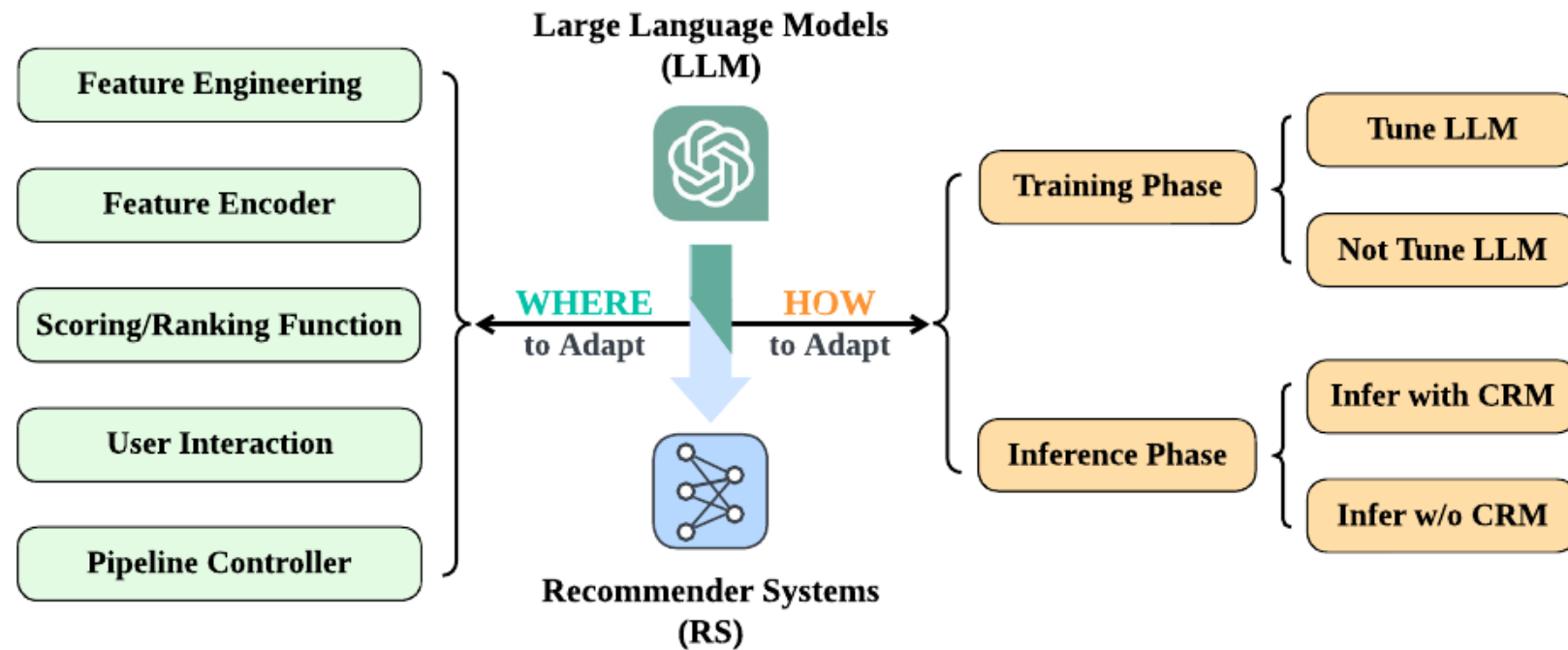


Fig. 1. The decomposition of our core research question about adapting LLM to RS. We analyze the question from two orthogonal perspectives: (1) where to adapt LLM, and (2) how to adapt LLM. Note that CRM stands for conventional recommendation model.

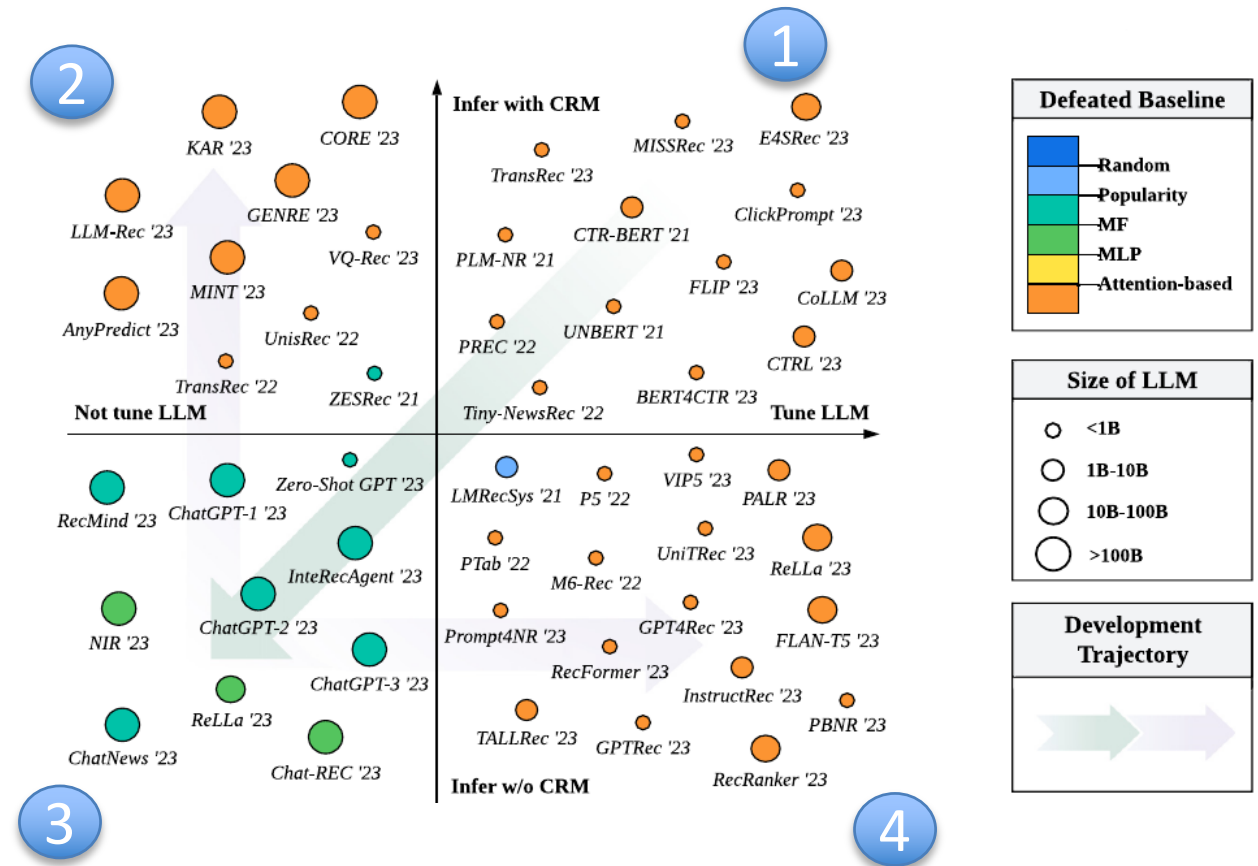
Dos preguntas ortogonales: **DÓNDE** (etapa del pipeline) y **CÓMO** (entrenamiento e inferencia)

DÓNDE: cinco roles del LLM en el pipeline

- **Feature engineering:** aumenta atributos de usuario/ítem y genera muestras de entrenamiento
- **Feature encoder:** mejora representaciones; recomendación cross-domain unificada
- **Scoring/ranking function:** puntúa ítems (pointwise), genera ítems o ambas tareas (híbrido)
- **User interaction:** interacción orientada a tareas o abierta (recomendación conversacional)
- **Pipeline controller:** el LLM orquesta herramientas y etapas del sistema (ej.: Chat-REC, RecLLM)

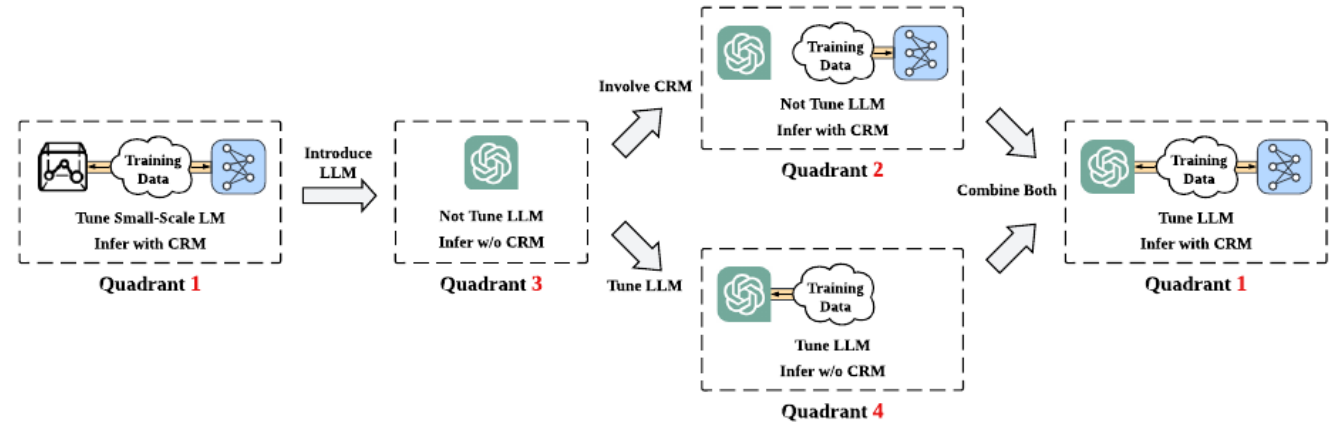
CÓMO: cuatro cuadrantes

- Criterio 1: ¿se **ajusta** (tune) el LLM con datos de recomendación? (full fine-tuning o PEFT como LoRA)
- Criterio 2: ¿se usa un modelo de recomendación convencional (**CRM**) en inferencia?
- Cuadrantes: (1) tune + CRM, (2) no tune + CRM, (3) no tune sin CRM, (4) tune sin CRM
- El tamaño y color de los círculos de la figura indican el tamaño del LLM y el mejor baseline superado



Lección del survey: hace falta conocimiento colaborativo

- Cuadrante 3 (LLM congelado, sin CRM) queda claramente por debajo, incluso con ChatGPT/GPT-4
- RecSys es un dominio especializado: el LLM no aprende señal colaborativa desde su corpus general
- Dos formas de inyectarla: **data-centric** (ajustar el LLM) y **model-centric** (inferir con CRM)
- Los LLMs funcionan bien como *re-rankers* de muestras difíciles: esquema **filter-then-rerank**



Beneficios de los LLMs para recomendación

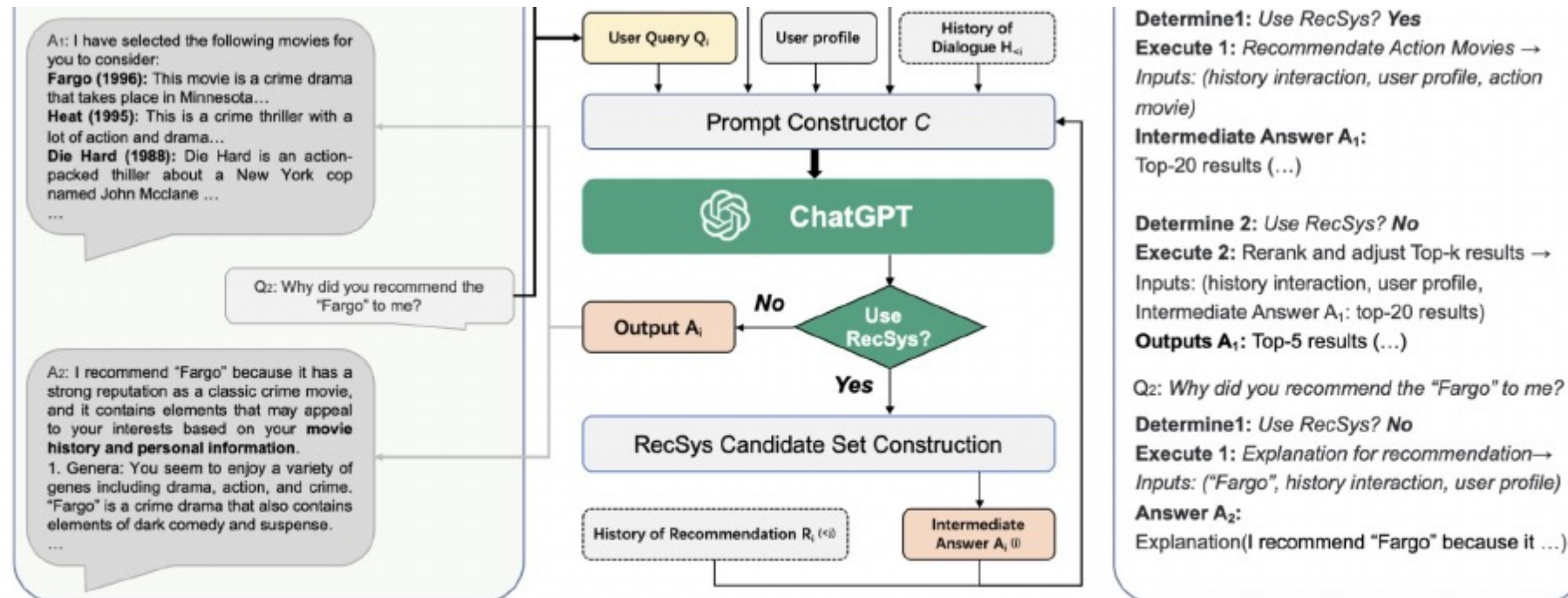
- **1. Conocimiento del mundo** (de la preentrenamiento): representan espacio y tiempo; embeddings de lenguaje útiles como representación de ítems
- **2. Comprensión y generación de lenguaje natural:** perfiles, explicaciones, interacción con el usuario
- **3. Comportamiento similar al humano:** razonamiento, seguimiento de instrucciones, simulación de usuarios

Primeros intentos y alineación

LLM COMO RECOMENDADOR SECUENCIAL

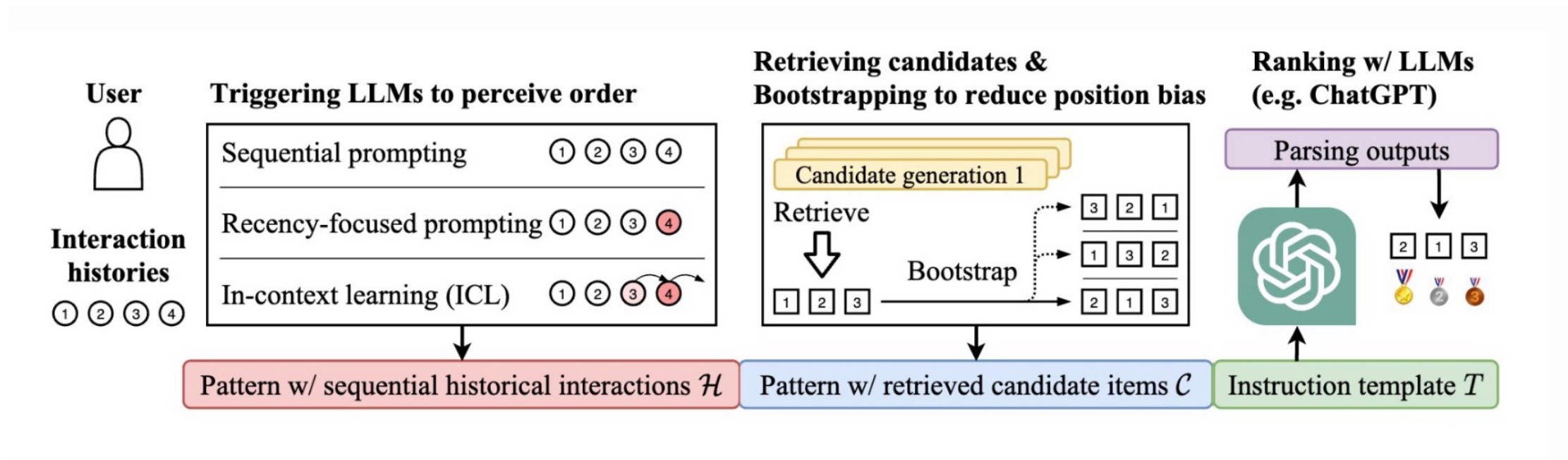
Primeros intentos: LLMs congelados + prompting

- Usar LLMs como GPT-4 sin ajustar: ingeniería de prompts + *in-context learning*
- **Chat-REC:** el LLM como controlador del recomendador



Primeros intentos: LLMs congelados + prompting

- Usar LLMs como GPT-4 sin ajustar: ingeniería de prompts + *in-context learning*
- **LLMRank**: LLMs como rankers zero-shot



LLMRank: Buen rendimiento sólo en zero-shot

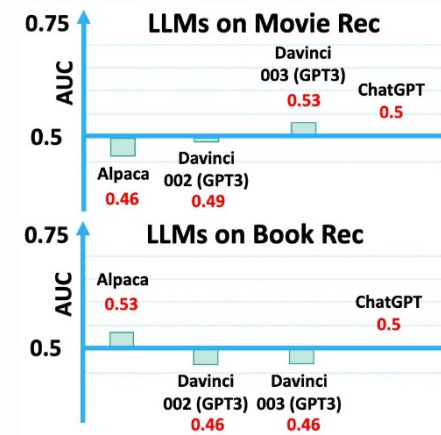
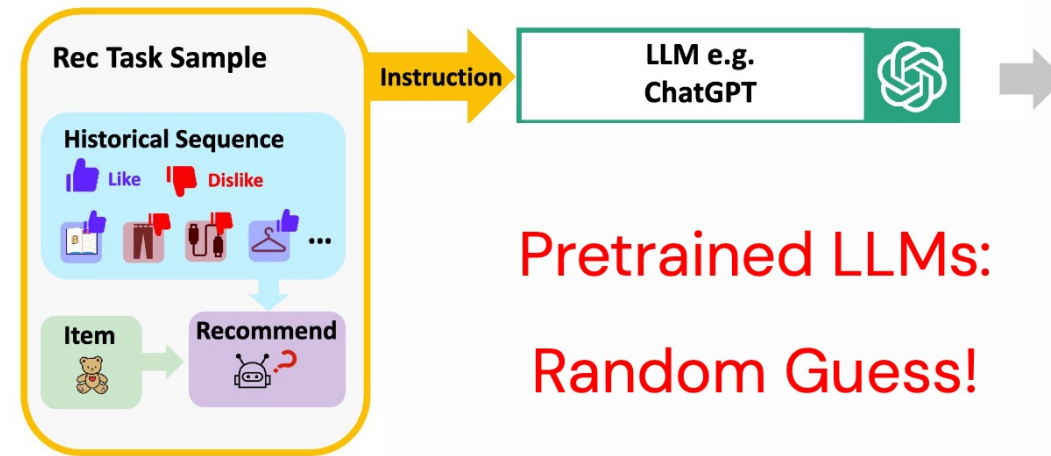
	Method	ML-1M				Games			
		N@1	N@5	N@10	N@20	N@1	N@5	N@10	N@20
full	Pop	0.08	1.20	4.13	5.79	0.13	1.00	2.27	2.62
	BPRMF [49]	0.26	1.69	4.41	6.04	0.55	1.98	2.96	3.19
	SASRec [33]	3.76	9.79	10.45	10.56	1.33	3.55	4.02	4.11
zero-shot	BM25 [50]	0.26	0.87	2.32	5.28	0.18	1.07	1.80	2.55
	UniSRec [30]	0.88	3.46	5.30	6.92	0.00	1.86	2.03	2.31
	VQ-Rec [29]	0.20	1.60	3.29	5.73	0.20	1.21	1.91	2.64
	Ours	1.74	5.22	6.91	7.90	0.90	2.26	2.80	3.08

Alinear LLMs con recomendación: cuatro variantes

- **1. Texto puro:** el ítem se representa por su descripción (TALLRec, P5)
- **2. + Embeddings colaborativos:** se inyectan embeddings de un CF (LLaRA, CoLLM, E4SRec)
- **3. + Tokens de ítems externos:** nuevos tokens para ítems, ej. Semantic IDs (CLLM4Rec, LC-Rec)
- **4. + Información multimodal:** imágenes y texto (VIP5)

Texto puro: TALLRec

- Formato de ítem: “atributo: valor”, usando solo el vocabulario del LLM
- Enfoque 1: sólo texto, LLM pre-entrenado



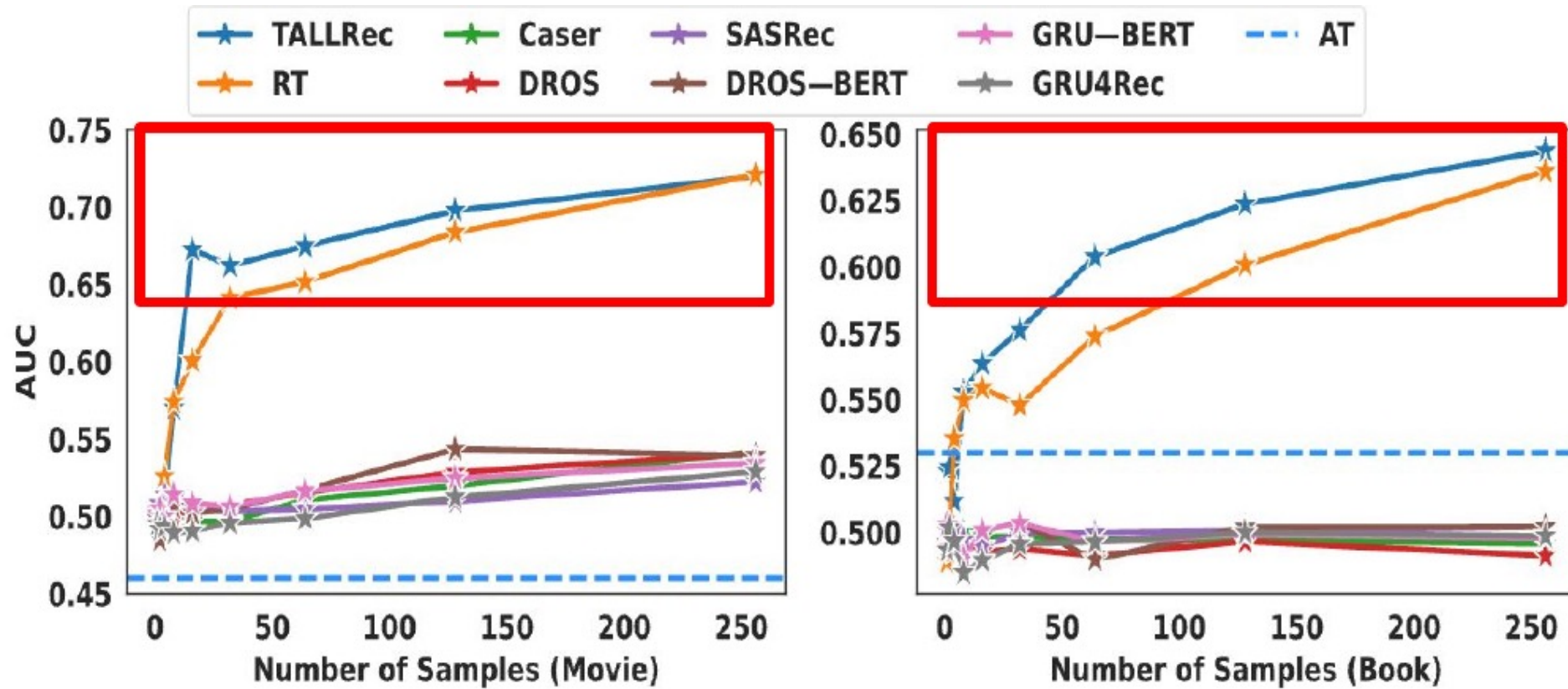
Texto puro con alineamiento: TALLRec

- Formato de ítem: “atributo: valor”, usando solo el vocabulario del LLM
- Ajuste en dos etapas: **Alpaca tuning** (seguir instrucciones) y **Rec-tuning** con LoRA
- Solo ~0,1% de parámetros; entrenable en una GPU de 24 GB
- Con 64 ejemplos: AUC 0,67 (películas) y 0,60 (libros), frente a $\approx 0,50$ de SASRec/GRU4Rec y ICL
- Generaliza de películas a libros



General task alignment → Recommendation alignment

TALLRec: pocos datos de ajuste, gran salto en AUC

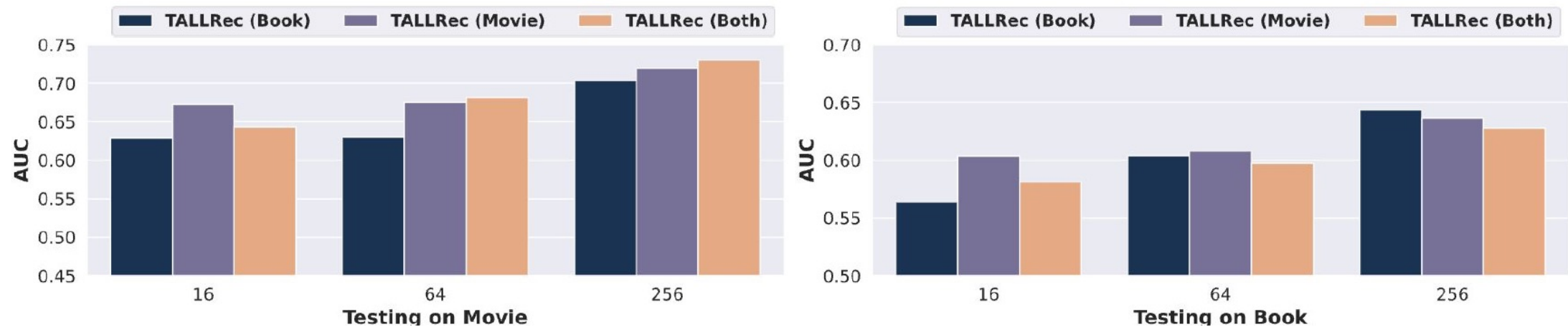


Eje x: número de ejemplos de ajuste. TALLRec (azul) despega con decenas de ejemplos; SASRec, Caser, GRU4Rec y DROS se quedan cerca de 0,5 en ese régimen.

Cómo leer ese gráfico sin exagerar la conclusión

- El régimen del experimento es **few-shot**: entre 16 y 256 ejemplos de ajuste
- Ahí el LLM parte con una ventaja enorme: ya sabe qué es *Matrix* y qué es *Inception*
- Un modelo como SASRec aprende el significado de un ítem **solo** desde las interacciones, y con 64 ejemplos no alcanza a aprender nada: por eso queda en 0,5
- Con datos abundantes la comparación cambia y los modelos secuenciales entrenados a escala vuelven a ser muy competitivos
- Conclusión honesta: los LLMs aportan **conocimiento previo**, no magia estadística

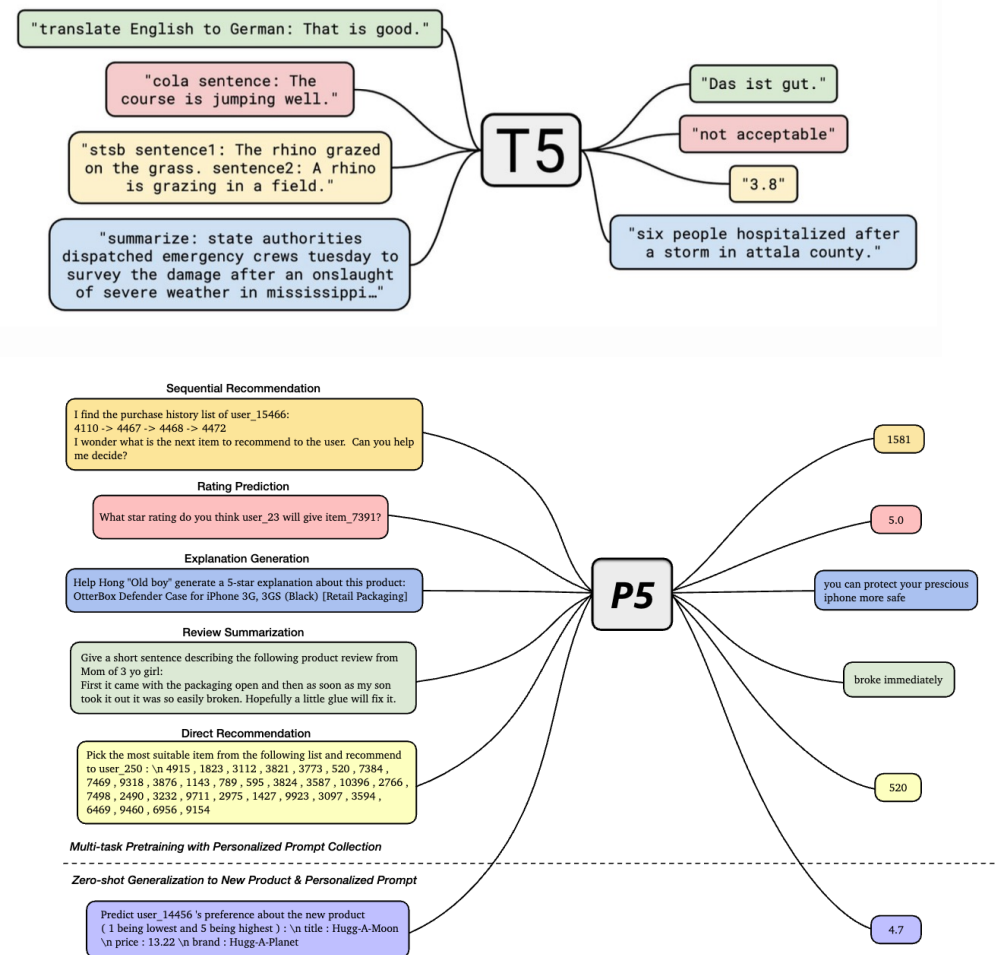
TALLRec: lo aprendido en un dominio sirve en otro



Ajustar en películas y evaluar en libros (y viceversa). Un recomendador basado en IDs no admite siquiera esta comparación: los ítems de un dominio no existen en el otro.

Texto puro: P5 – todo es lenguaje

- Convierte tareas (rating, secuencial, explicación, resumen de reviews, búsqueda...) en **plantillas de prompt personalizadas**
- Una única arquitectura **encoder-decoder (T5)** y un único objetivo de *language modeling* (texto a texto)
- Elimina las tablas de embeddings de IDs dispersos
- Iguala o supera baselines específicos en 5 familias de tareas; transfiere entre dominios (ej.: Toys → Beauty)



P5: un modelo texto-a-texto frente a baselines específicos

Table 3: Performance comparison on sequential recommendation.

Methods	Sports				Beauty				Toys			
	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10
Caser	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0463	0.0306	0.0675	0.0374
S ³ -Rec	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
P5-S (2-3)	0.0272	0.0169	0.0361	0.0198	<u>0.0503</u>	<u>0.0370</u>	<u>0.0659</u>	<u>0.0421</u>	0.0648	0.0567	0.0709	0.0587
P5-B (2-3)	<u>0.0364</u>	<u>0.0296</u>	<u>0.0431</u>	<u>0.0318</u>	0.0508	0.0379	0.0664	0.0429	0.0608	0.0507	0.0688	0.0534
P5-S (2-13)	0.0258	0.0159	0.0346	0.0188	0.0490	0.0358	0.0646	0.0409	<u>0.0647</u>	<u>0.0566</u>	<u>0.0705</u>	<u>0.0585</u>
P5-B (2-13)	0.0387	0.0312	0.0460	0.0336	0.0493	0.0367	0.0645	0.0416	0.0587	0.0486	0.0675	0.0536

Recomendación secuencial en Sports, Beauty y Toys. P5 (últimas filas) compite con Caser, HGN, GRU4Rec, BERT4Rec, FDSA, SASRec y S³-Rec usando un único modelo y un único objetivo.

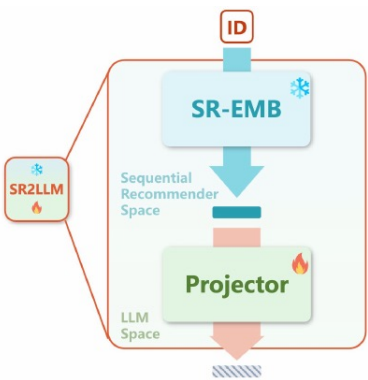
Advertencia metodológica sobre estas comparaciones

- Varios resultados tempranos se evaluaron con **muestreo de negativos** (ordenar el ítem correcto contra 99 al azar), que infla las métricas respecto de ordenar todo el catálogo
- Algunos trabajos restringen los candidatos a una lista que **ya contiene** la respuesta correcta
- Por eso conviene comparar siempre: mismo protocolo, mismo conjunto de candidatos, mismo *split*
- Esto no invalida la idea de GenRec; invalida leer las tablas como un ranking definitivo
- Es el mismo cuidado metodológico que vimos en la unidad de evaluación del curso

Embeddings colaborativos: LLaRA y CoLLM

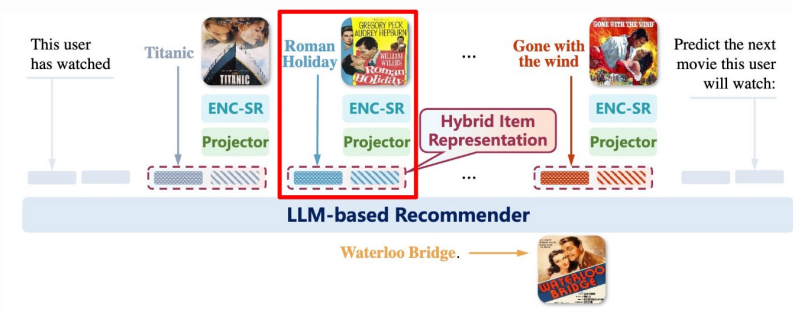
- Motivación: el *language modeling* no captura señal colaborativa (como en NGCF)
- **LLaRA**: prompt híbrido con texto + embeddings de ítems preentrenados (de un recomendador secuencial) proyectados al espacio del LLM

(a) Text-only prompting method.	(b) Hybrid prompting method.
Input: This user has watched Titanic [PH], Roman Holiday [PH], Gone with the wind [PH] in the previous. Please predict the next movie this user will watch. The movie title candidates are The Wizard of Oz [PH], Braveheart [PH],..., Waterloo Bridge [PH],... Batman & Robin [PH]. Choose only one movie from the candidates. The answer is	Input: This user has watched Titanic [emb _s ¹⁴], Roman Holiday [emb _s ²⁰], Gone with the wind [emb _s ²⁷] in the previous. Please predict the next movie this user will watch. The movie title candidates are The Wizard of Oz [emb _s ²], Braveheart [emb _s ⁴²],..., Waterloo Bridge [emb _s ²⁰],... Batman & Robin [emb _s ¹⁹]. Choose only one movie from the candidates. The answer is
Output: Waterloo Bridge.	Output: Waterloo Bridge.



(2) + Collaborative embeddings (LLaRA)

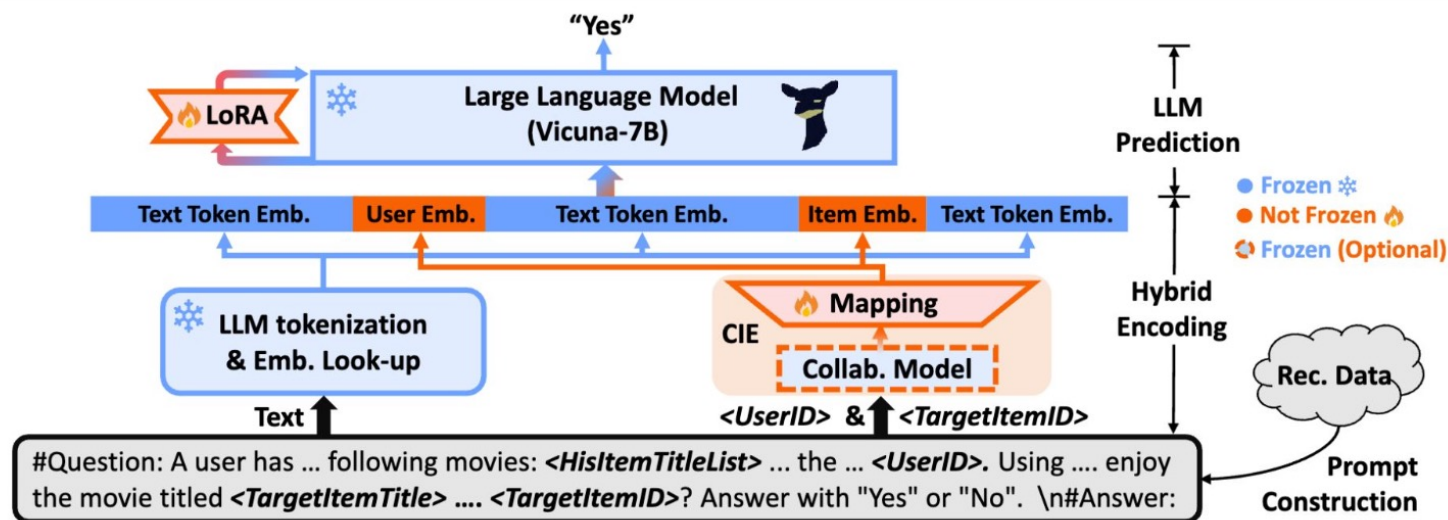
+ Pretrained item embeddings



Embeddings colaborativos: LLaRA y CoLLM

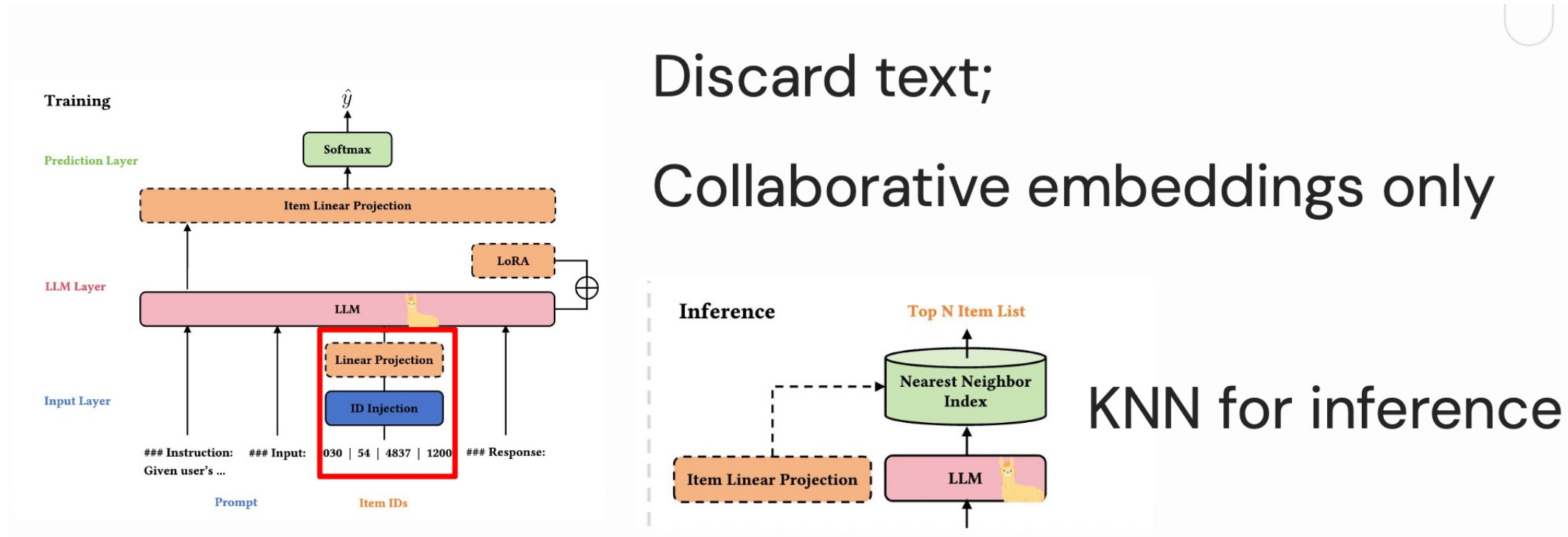
- Motivación: el *language modeling* no captura señal colaborativa (como en NGCF)
- **CoLLM**: además de ítems, inyecta embeddings de usuario

Pretrained item embeddings + user embeddings



E4sRec

- Motivación: el *language modeling* no captura señal colaborativa (como en NGCF)
- **E4SRec**: usa embeddings de IDs de un modelo secuencial

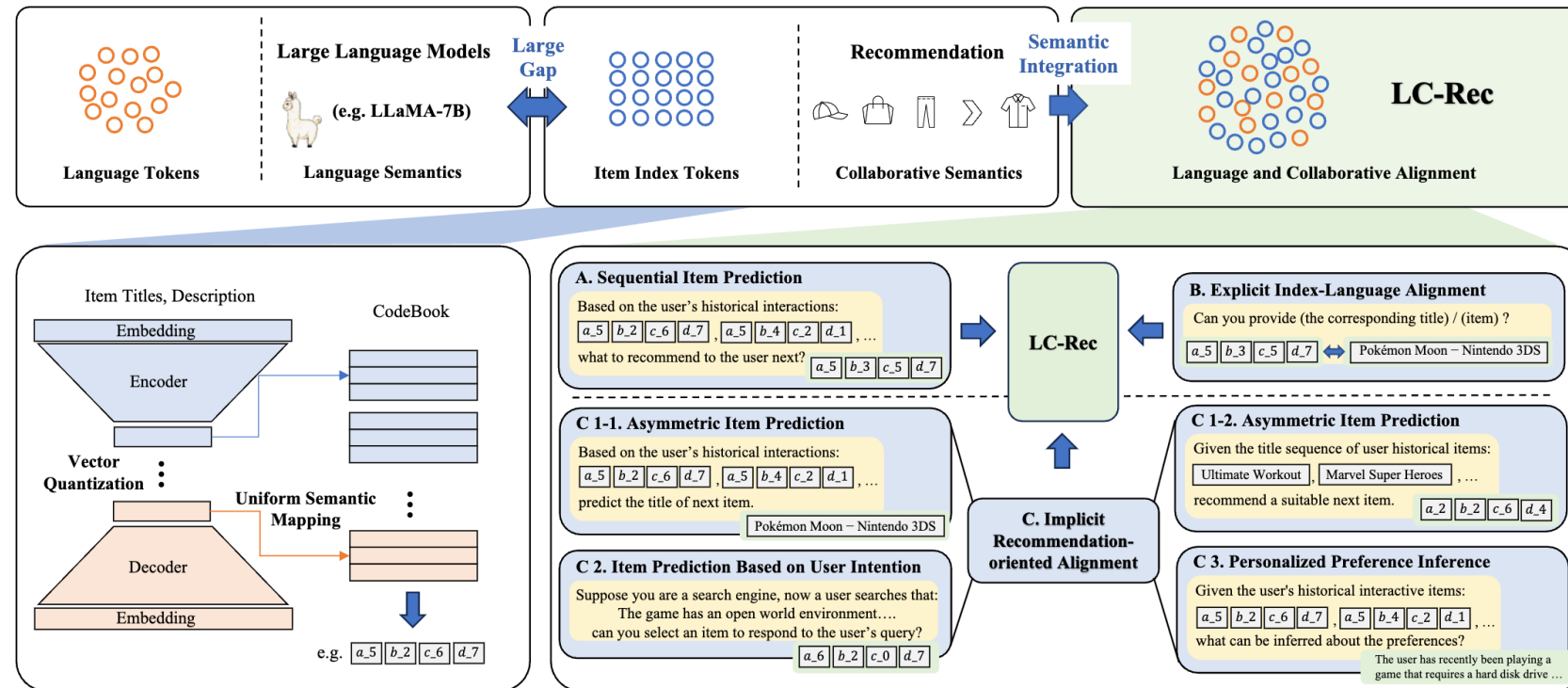


Fuente: Tutorial 'Large Generative Recommendation Models' (WWW 2025) (pp. 59-64)

Li et al. E4SRec: An Elegant Effective Efficient Extensible Solution of Large Language Models for Sequential Recommendation. WWW 2024.

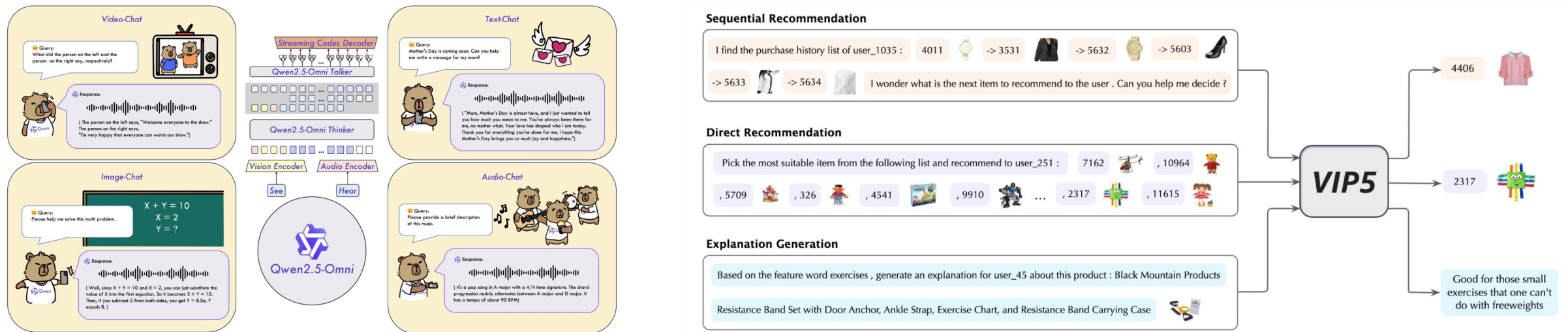
Tokens externos de ítem: LC-Rec

- Los tokens del lenguaje no son óptimos para indexar ítems
- **LC-Rec** agrega **Semantic IDs** (ítems similares tienen IDs similares) al vocabulario del LLM
- Tareas de alineación entre lenguaje y semántica colaborativa: predicción secuencial de ítems, traducción título ↔ SemID



Información multimodal: VIP5

- LLMs actuales en realidad son MLLMs
- **Se puede usar información multimodal: Diferencia con P5.**



Gen et al. VIP5: Towards Multimodal Foundation Models for Recommendation. EMNLP 2023 (Findings).

Fuente: Tutorial 'Large Generative Recommendation Models' (WWW 2025) (pp. 66-70, 238)

Comparación de enfoques de adaptación

Enfoque	Representación del ítem	Señal colaborativa	Ejemplos
Prompting congelado	Texto	No	Chat-REC, LLMRank
Texto puro + tuning	Texto	Solo implícita en el tuning	TALLRec, P5
+ Embeddings CF	Texto + embedding	Sí (modelo CF externo)	LLaRA, CoLLM, E4SRec
+ Tokens externos	Semantic IDs / tokens nuevos	Sí (en la tokenización)	LC-Rec, CLLM4Rec
+ Multimodal	Texto + imagen	Según variante	VIP5

Fuente: Tutorial 'Large Generative Recommendation Models' (WWW 2025); Lin et al. How Can Recommender Systems Benefit from Large Language Models: A Survey. ACM TOIS 43(2), 2025.

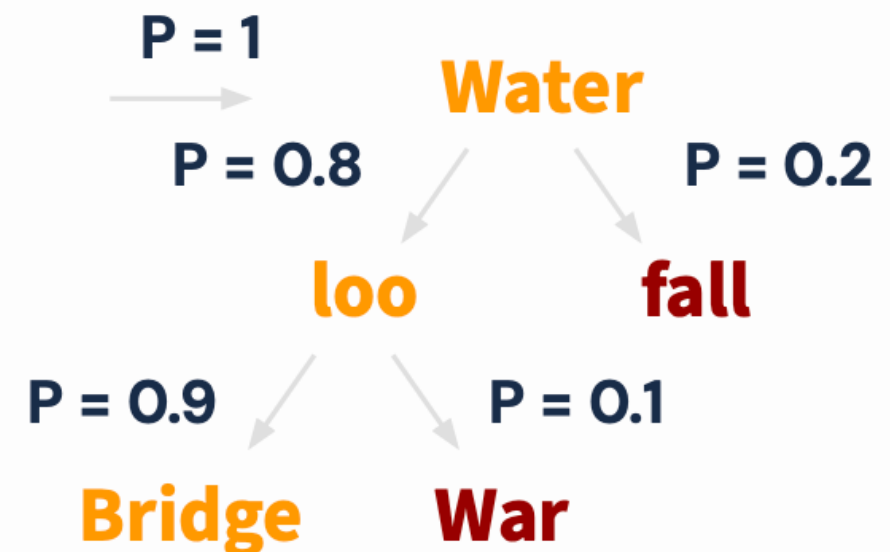
Objetivos de entrenamiento

- **Supervised fine-tuning (SFT):** prompt con historial (“He visto Titanic, Roman Holiday... Predice la próxima película”) y objetivo “Waterloo Bridge”
- **Aprendizaje de preferencias:** los LLMs se alinean con preferencias humanas y la recomendación trata de preferencias → variantes de DPO (ej.: S-DPO, clase 3)
- **Aprendizaje por refuerzo:** inspirado en DeepSeek-R1; **Rec-R1** maximiza la recompensa que entrega el propio recomendador

Inferencia: generar ítems válidos

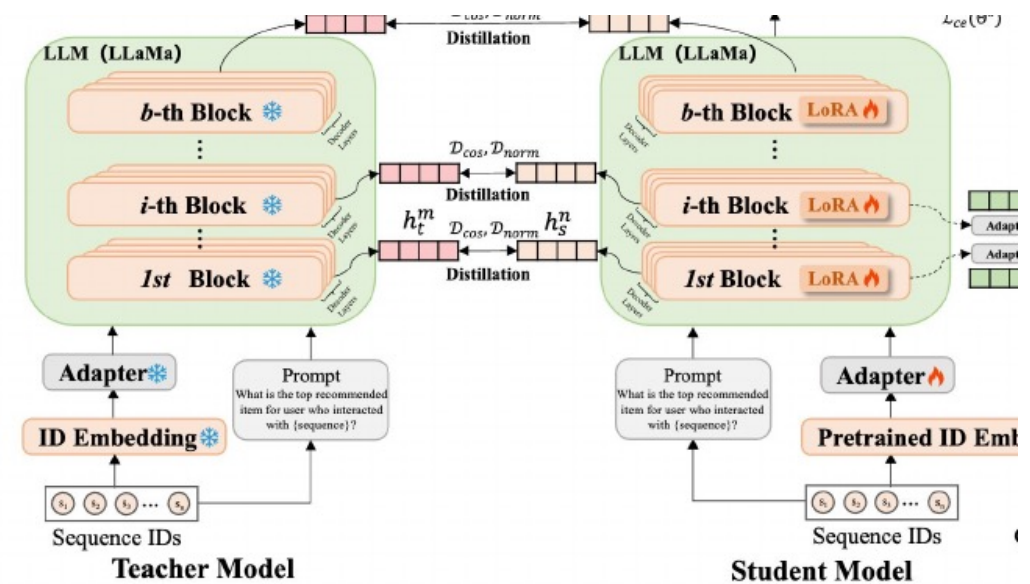
- **Beam search:** mantiene las k secuencias con mayor probabilidad
- Problema: el LLM puede generar títulos que no existen en el catálogo
- **Constrained beam search:** restringe la generación con una estructura (trie) de ítems válidos; en cada paso solo se permiten tokens que continúan algún ítem real
- Diseños especiales: la *length penalty* es redundante en recomendación; sesgo de amplificación y homogeneidad (Bao et al., EMNLP 2024)

I have watched Titanic, Roman Holiday, ...
Gone with the wind. Predict the next movie
I will watch:



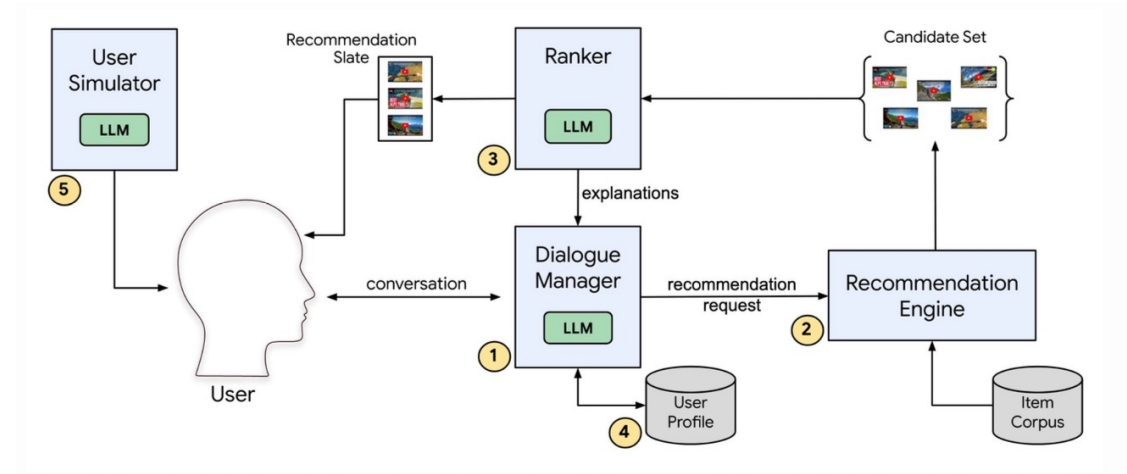
Eficiencia: un requisito de despliegue

- **Entrenamiento:** menos es más (LIMA: ~1k ejemplos de alta calidad); selección de datos
- **Inferencia:** la decodificación autoregresiva domina el tiempo; decodificación especulativa para GenRec (Lin et al., ICLR 2025)
- **Tamaño:** poda (LLM-Pruner; rendimiento similar con 0,6% de parámetros reportado en M6-Rec) y destilación (SLMRec: modelo pequeño aprende del LLM con etiquetas duras y blandas)

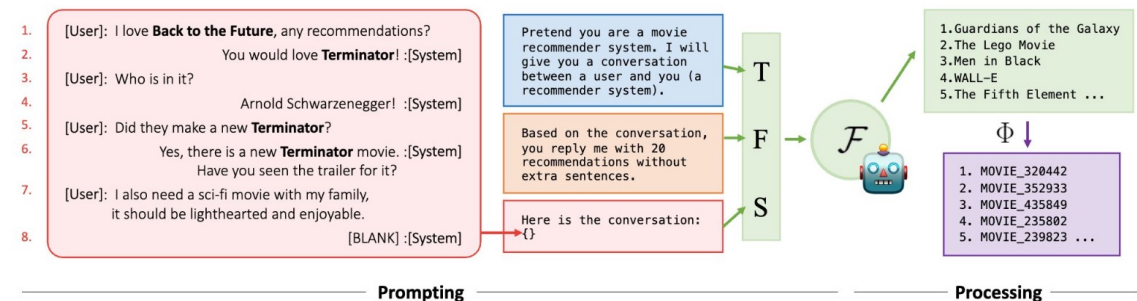


LLMs como recomendadores conversacionales

- **CRS clásicos:** basados en atributos o de forma libre; modelos específicos de tarea (ej.: KBRD)
- **LLM como CRS:** marco RecLLM; LLMs como CRS zero-shot
- Mejoras: demostraciones, + grafo de conocimiento, + información colaborativa



Friedman et al. Leveraging Large Language Models in Conversational Recommender Systems. arXiv:2305.07961.



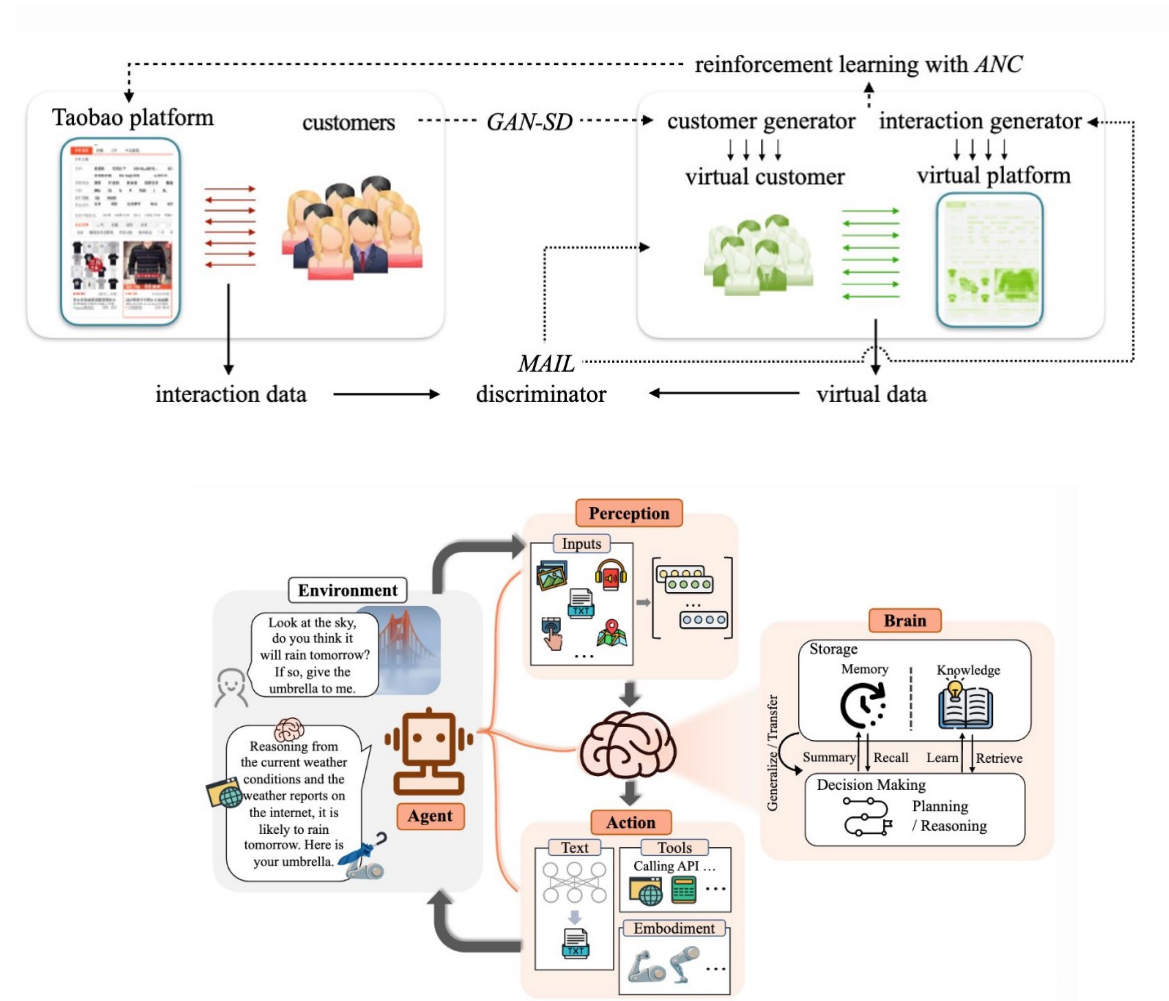
He et al. Large Language Models as Zero-Shot Conversational Recommenders. CIKM 2023.

Desafíos de recomendadores conversacionales

- Datasets:
 - Public datasets for CRS are limited, due to the scarcity of conversational products and real-world CRS datasets
- Evaluación (incluida evaluación vía LLMs):
 - Traditional metrics like NDCG and BLEU are often insufficient to assess user experience
- Producto:
 - What is the form of LLM-based CRS products? ChatBot? Search bar? Independent App?

LLMs como simuladores de usuario

- Simuladores previos (basados en RL) tienen un alcance limitado
- **Agentes generativos para recomendación:** agentes con perfil, memoria y acciones que simulan usuarios
- Extensiones: comportamientos sociales, conversación multi-turno, objetivos de simulación multifacéticos
- Uso: evaluar y entrenar recomendadores sin usuarios reales (con cautela sobre fidelidad)



Desafíos al llevar LLMs a RecSys real

Eficiencia	Efectividad	Ética y riesgos
Costo de entrenamiento	Texto largo en el dominio (historiales)	Equidad (fairness)
Latencia de inferencia	Indexación y modelamiento de IDs	Alucinaciones
Tamaño del modelo	Conocimiento colaborativo vs. conocimiento general	Privacidad y explicabilidad

Para discutir

- ¿Por qué un LLM congelado rinde peor que SASRec pese a tener “conocimiento del mundo”?
- ¿Qué se pierde y qué se gana al representar un ítem solo por su texto?
- ¿Cómo garantizar que lo generado sea un ítem real del catálogo?
- ¿Cuándo vale la pena el costo de un LLM frente a un modelo pequeño con conocimiento colaborativo?

Resumen y próxima clase

- La cascada discriminativa tiene límites estructurales; GenRec propone **generar** ítems
- Adaptar LLMs requiere **conocimiento colaborativo** (ajuste y/o CRM)
- Variantes: texto, + embeddings CF, + tokens de ítem, + multimodal
- Quedan abiertas: **¿cómo representar millones de ítems con pocos tokens?**
- **Próxima clase:** tokenización de ítems – RQ-VAE y Semantic IDs

Referencias

- Lin et al. How Can Recommender Systems Benefit from LLMs: A Survey. ACM TOIS, 2025.
- Li et al. Tutorial on Generative Recommendation: Foundations and Frontiers. KDD 2026.
- Large Generative Recommendation Models (tutorial). WWW 2025.
- Li et al. A Survey of Generative Recommendation from a Tri-Decoupled Perspective. 2025.
- Bao et al. TALLRec. RecSys 2023. Geng et al. P5. RecSys 2022. Liao et al. LLaRA. SIGIR 2024.
- Zhang et al. CoLLM. TKDE 2025. Zheng et al. LC-Rec. ICDE 2024. Zhai et al. HSTU. ICML 2024.
- Gao et al. Chat-REC. 2023. Hou et al. LLMRank. ECIR 2024. Lin et al. Rec-R1. 2025.

Fin

- `denis.parra@uc.cl`